Mathora: Learning Platform for Mathematics

INFS3208 Individual Project Proposal

Details

Name	Jonah Benedicto
Student Number	48081243
Date	21/10/2026
Type	Type I: Highly Scalable and Available Web Application

Introduction

Mathora is a web application that gives Australian school students a free and structured way to practise mathematics, and gives software developers a simple way to add mathematics questions to their own applications. Every question in Mathora is linked to a specific topic in the Australian Curriculum: Mathematics (Foundation to Year 10) or one of the four Queensland senior mathematics syllabuses, so students always know which part of the curriculum they are practising. Answers are marked instantly, and the marking understands mathematics rather than simply comparing text. The motivation for this project comes from two gaps. First, there is currently no freely available, curriculum-linked collection of mathematics questions that students can use to learn on their own. Students mainly rely on textbooks and commercial practice websites, but textbooks are expensive, and both often contain a large amount of extra material that makes it harder to find focused practice on one topic. Second, there is no simple tool that lets developers integrate curriculum-aligned mathematics questions and marking into their own applications. A developer building a study or tutoring app would have to write their own questions and marking logic, which is time-consuming and error-prone, especially for algebra, where many different answers can be correct. The objective of Mathora is to solve both problems with one reliable platform. Its features include user accounts with secure login and logout, practice sessions where students choose a course and topic and receive instant feedback, quizzes that can be created, submitted and reviewed, and progress tracking that shows a student's accuracy by course and topic and recommends topics to practise next. Students can also search the full curriculum. A separate developer site provides a public, rate-limited API for the questions, answer marking, a mathematics solver and the curriculum, together with interactive documentation. Traditional computing solutions do not suit this problem well. If Mathora ran on a single on-premises server, its capacity would be fixed. The server could become overloaded on school nights and before exams, but would sit mostly idle during school holidays, and the hardware would have to be bought for the busiest periods. A single server is also a single point of failure, so any hardware fault or software update would take the whole application offline. Finally, a single monolithic application cannot scale its busiest parts, such as answer marking, independently of the rest. Cloud computing addresses each of these limitations. Resources can be added within minutes when demand rises and removed when it falls, so the project only pays for what it uses. By splitting the application into microservices running in containers, each part can be scaled independently. Kubernetes automatically restarts failed containers, spreads copies of each service across several machines, balances requests between them, and replaces software versions gradually with rolling updates that can be rolled back if a problem appears. Managed services such as Google Kubernetes Engine also remove the need to maintain physical servers.

Technical Solutions

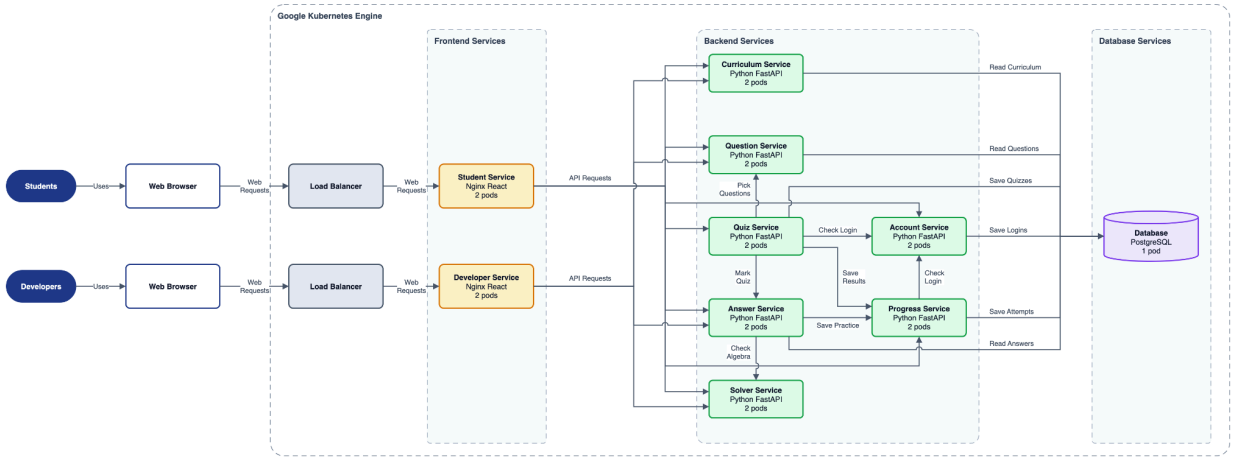
The frontend consists of two single-page web applications built with React and TypeScript, bundled with Vite and styled with Tailwind CSS. The student site uses React Router for navigation and KaTeX to display mathematical notation. The developer site uses Swagger UI to present interactive API documentation. Each site is served by Nginx, which also acts as a gateway that forwards API requests to the backend. The developer site's gateway only exposes the four public services, removes login tokens and limits each IP address to ten requests per second. The backend is made up of seven Python microservices built with FastAPI and run by Uvicorn. The Account service handles registration, login and sessions; the Curriculum and Question services provide curriculum and question data; the Answer service marks answers; the Solver service performs symbolic mathematics with SymPy; the Quiz service creates and marks quizzes; and the Progress service records attempts and produces recommendations. The services communicate through REST requests. Data is stored in a PostgreSQL 17 database with eleven tables and around 19,500 records, kept on a persistent disk so that it survives restarts. Every service runs

in its own Docker container, giving ten containers in total. Docker Compose runs the whole system locally for testing, and the images are stored in Google Artifact Registry. The application is deployed on a Google Kubernetes Engine cluster in Sydney with three n1-standard-2 nodes. Each service runs two replicas with readiness checks and a short shutdown delay, so that scaling and updates do not interrupt users. Monthly costs were estimated with the Google Cloud Pricing Calculator in Australian dollars for the Sydney region, running continuously.

Google Pricing Calculator	**Monthly Cost**
Kubernetes Engine	A$552.94
Persistent Disk	A$1.92
Networking, Cloud Load Balancing	A$35.33
Networking, Data Transfer	A$5.41
Artifact Registry	A$0.36
Total	≈ A$606

The GKE free tier covers the A$103.91 cluster fee for one zonal cluster. During development the cluster is paused by resizing it to zero nodes, so a month with 40 hours of testing costs roughly A$70.

Architecture Design



Students and developers each reach their own site through a separate load balancer, which passes requests to one of that site's two Nginx pods. Nginx returns the web application and forwards API requests to the backend, where each Kubernetes Service balances requests across its pods. The student site can reach all seven services, while the developer site can only reach the Curriculum, Question, Answer and Solver services. The Quiz service picks questions from the Question service, has answers marked by the Answer service and saves results through the Progress service. The Answer service sends algebra to the Solver and records practice attempts with Progress, while Quiz and Progress check logins with the Account service. Every service except the Solver reads from or saves to the PostgreSQL database.